Sketchpad, Then and Next

A guide to Version 2 and its drawing model

For Barry Kingsbury · 1 November 2026

Companion to *Sketchpad Version 2 Functional Specification* (SPV2-FS-001)

Contents

How this guide is organised 3
Part 1. Sketchpad today 4
1.1 The pieces and how they connect 4
1.2 How a drawing gets onto the screen 4
1.3 How Figures holds layers and welds 5
Part 2. What gets in the way 6
2.1 The shape exists only while it is being painted 6
2.2 One thing, many owners 6
2.3 Every change is final 6
2.4 Adding something takes bookkeeping 6
2.5 Scripts share one room 7
2.6 Two kinds of drawing, two kinds of code 7
2.7 Small things that add up 7
Part 3. The new design, from the top down 8
Cycle 1. The whole in one picture 8
Cycle 2. The blocks 8
Cycle 3. Three actions, followed through the blocks 9
Part 4. The drawing model: a tutorial 12
4.0 The whole model on one page 12
Lesson 1. A path is a list of points 12
Lesson 2. Connecting two paths: the parabola 13
Lesson 3. Direction and length matter 13
Lesson 4. The same connector on other paths 14
Lesson 5. One closed path, joined to itself 14
Lesson 6. Shapers: changing a path before it is connected 16
Lesson 7. Where paths come from: makers, and why parametric is not special 17
Lesson 8. Finding points worth naming 18
Lesson 9. Layers that use other layers 18
Lesson 10. Putting it together: a figure with no code 20
Lesson 11. Writing your own generator 20
Lesson 12. How today's objects come out 22
Lesson 13. What the model does not cover 22
Quick reference: the kernel's vocabulary 23

How this guide is organised

The specification says what Version 2 must do. This guide is for understanding it. Its pieces don't make sense one at a time: the drawing model only makes sense alongside the document it lives in, and the document only alongside the blocks around it. So the guide goes over the whole thing more than once, each pass closer in:

1. **Part 1, Sketchpad today.** How the current program is put together, with diagrams, so the changes have something to be measured against.
2. **Part 2, What gets in the way.** The weaknesses, in plain terms, grouped by cause rather than listed by file.
3. **Part 3, The new design from the top down.** Three cycles. The first is one picture of the whole. The second names the blocks and what each is for. The third follows three ordinary actions through the blocks. Blocks such as the catalog and storage stop at that level: what they do, not how.
4. **Part 4, The drawing model: a tutorial.** The part you will live with. It starts from a single list of points and builds, lesson by lesson, to figures whose layers depend on each other and to writing your own object. Every picture in Part 4 was computed from the rules it illustrates, so what you see is exactly what the rule produces.

Words in `this type` are names as they would appear in code. The kernel's vocabulary is summarised on the last page for reference while you read.

Part 1. Sketchpad today

1.1 The pieces and how they connect

Sketchpad today is six tabs over a shared pool of memory. Each tab reads and writes two shared state objects, `uiState` and `drawState`. Under them sit three kinds of drawing code:

- the 14 Draw registry objects;
- the Figures code, which arranges registry objects as layers;
- the script runner, which runs the pattern and gallery scripts.

All three end in the same place: the drawing helpers in `draw/` and `classes/`, which finish in `drawLine`, which paints onto the canvas. A separate Node program does every file write and keeps the manifests (the lists of what is in each folder) up to date.

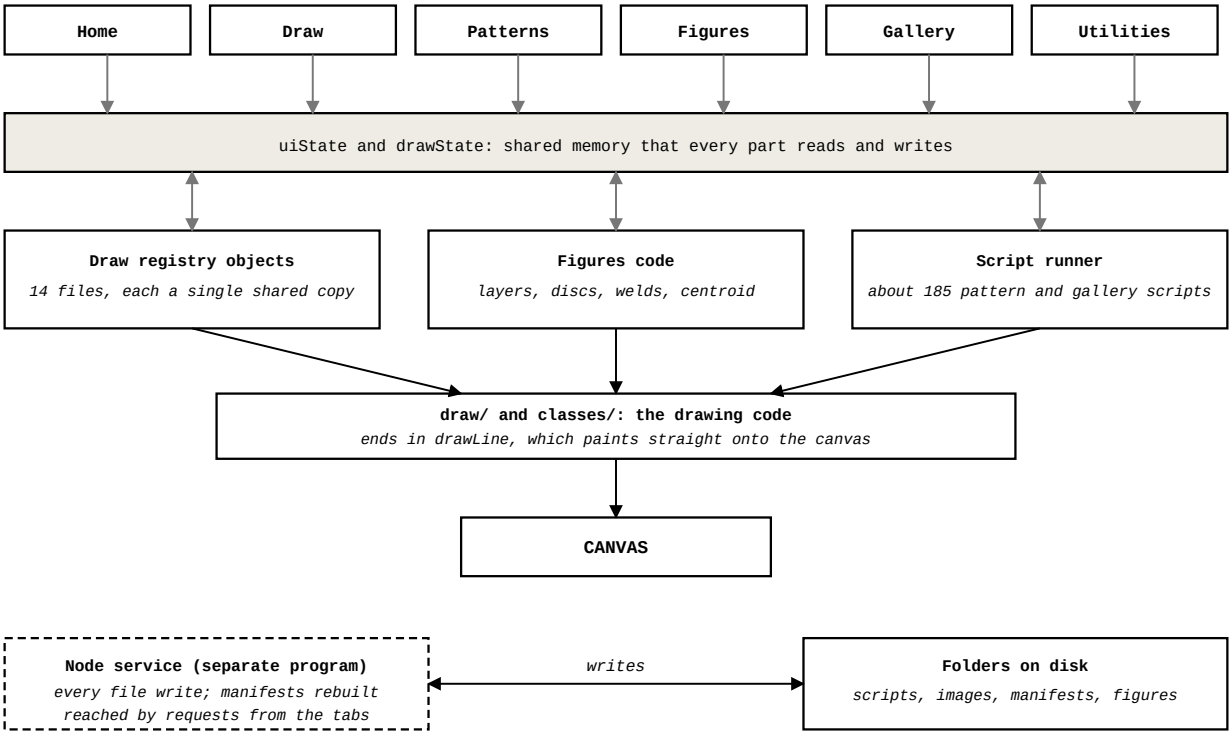


Figure 1. Sketchpad today: six tabs over shared memory, three kinds of drawing code, one canvas

1.2 How a drawing gets onto the screen

Take the Parabola in the Draw tab. Its settings live in `params`: the four arm ends, the step count, the colour, and also `points[0..5]`, a second copy of the end points that the drag code uses. When you move a slider or drag a red dot, `update()` compares the copies to work out which point you moved, rebuilds its live objects (a `Line` and a `StringThing`), and `draw()` calls the helpers, which call `drawLine` once per stitch.

Two things in Figure 2 matter for what follows. The first is that the stitches never exist as a list anywhere: each one is computed and painted in the same moment, inside `drawLine`. The second is that when Phase 7d needed

to know where discs could go, the only way was to add `infrastructure()`, a second description of the same shape, which has to be kept in agreement with `draw()` by hand.

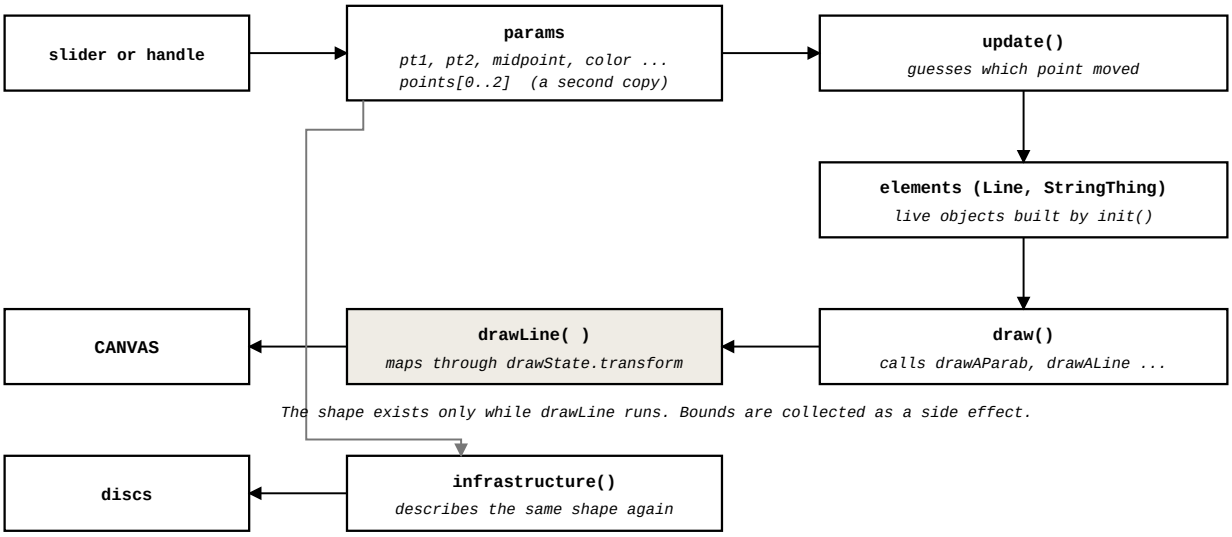


Figure 2. One drawing, today: settings, a guess, live objects, and painting

1.3 How Figures holds layers and welds

A figure today is an array of layers plus a parallel array of live objects, one per layer. Each layer's object keeps its own copy of its handle positions. A weld is a list of discs that ought to sit at the same spot. Because each layer holds its own copy of that spot, a drag on one layer has to be followed by work to bring the others along, as Figure 3 shows. That work is the propagation walk: four special cases, plus conflicts refused partway through a drag.

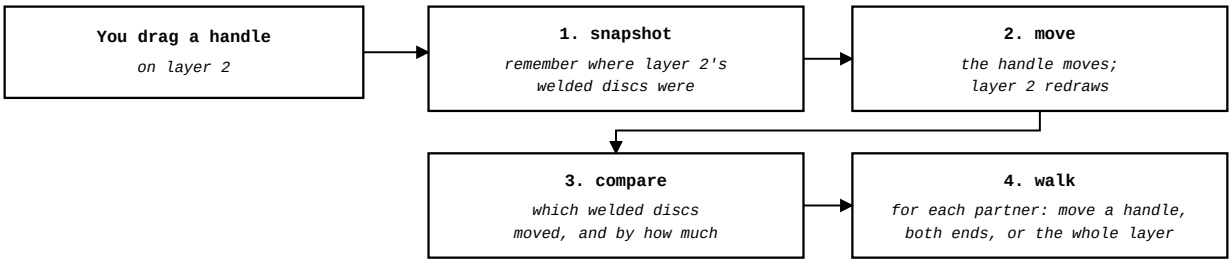


Figure 3. A drag on a welded layer, today

Part 2. What gets in the way

The program works and its tests pass. The weaknesses below aren't bugs; they're the reasons each new feature costs more than the last. They are grouped by cause.

2.1 The shape exists only while it is being painted

Because stitches are computed and painted in one step, anything else that needs the shape has to get it some other way:

- **discs** through `infrastructure()`;
- **bounds** collected as a side effect while drawing;
- **thumbnails** by borrowing the main canvas;
- **a printed template** would have to be drawn all over again.

Every one of these is a second account of the same geometry.

2.2 One thing, many owners

A registry object is a single shared copy, so Draw, Figures and Patterns end up sharing one set of settings. Inside an object, end points are kept twice: once by name and once in `points[]`. Across a figure, a welded spot is kept once per layer. Wherever there are copies, code has to keep them in step, and that's where the bugs have been, including both Phase 8 bugs found in real use.

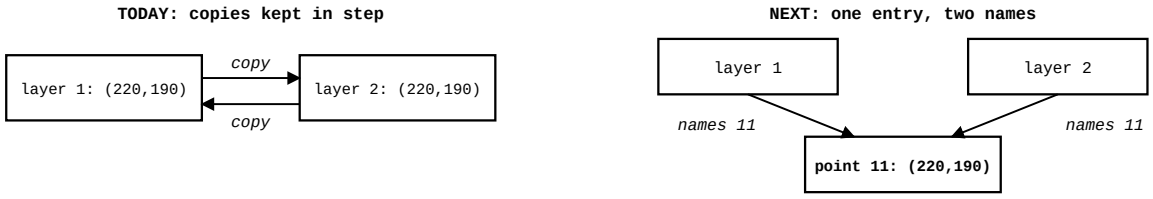


Figure 4. Copies kept in step, against one entry with two names

2.3 Every change is final

There is no undo. Delete Layer warns that it can't be taken back. In a program meant for sketching, where trying something and backing out is the point, that's a real cost.

2.4 Adding something takes bookkeeping

A new script shows up only after a restart or a Rebuild, because its folder's manifest has to be rewritten. A new Draw object needs a line added by hand to `drawRegistry.js`. `boxes.js` shows what happens when that step is missed: it's loaded and never appears, and nothing says why. The manifests are copies of what's in the folders, so they can disagree with the folders. That's why there are start-up rebuilds, missing-file checks and an alert bar.

2.5 Scripts share one room

Every script draws on the same canvas with the same shared state. One script can leave something behind that the next one trips on. Two scripts fail after about a hundred others have run.

2.6 Two kinds of drawing, two kinds of code

Parametric drawings live in their own code (`drawParametrics.js`). The good tools for spacing points (even spacing by arc length, bias, jitter, all in `createNodes`) work only for ellipses. A rose curve can't use them.

2.7 Small things that add up

Controls are declared in two blocks, with widget names instead of value types, and with two names for the colour widget. "Overlay" means three different things. Each tab builds its menus its own way. None of these is serious alone; together they make every change start with "which convention does this part use?"

Part 3. The new design, from the top down

Cycle 1. The whole in one picture

Version 2 runs left to right in four stages (Figure 5). **Files** hold the generators (one file per drawable object or script) and your saved work. A **document** records what you made: which generators, with which settings, where, and which points they share. The document is turned into **geometry**: paths, stitch lines and named points, all as plain data. Geometry becomes **output**: the screen, a thumbnail, or a printed template.

You change the document and nothing else, by commands: drag, slide, weld, undo. Under the geometry stage sits the **kernel**, the small vocabulary every drawing is written in. Part 4 is about that vocabulary.

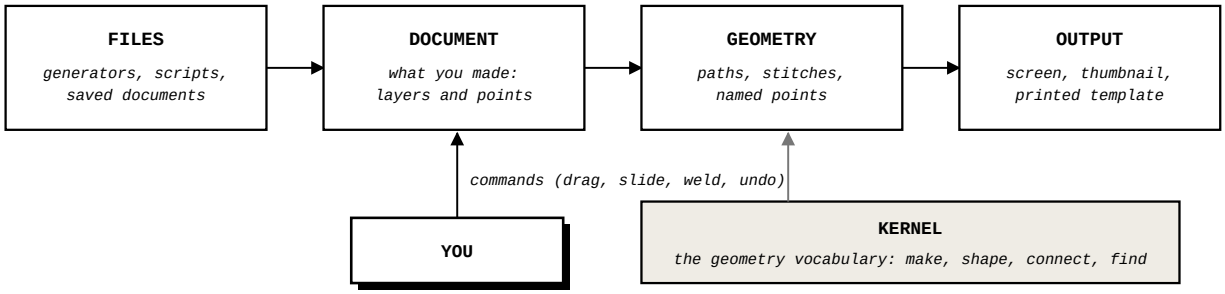


Figure 5. Cycle 1: files, document, geometry, output

Three things are kept strictly apart. The **kernel** is geometry only: values in, values out, no canvas and no memory. **Generators** describe objects and hold no state. **Documents** are plain data and hold no code. Most of Part 2's weaknesses come from these three being mixed.

Cycle 2. The blocks

Figure 6 opens up the four stages into the blocks that do the work. Arrows show which block uses which; none points upward.

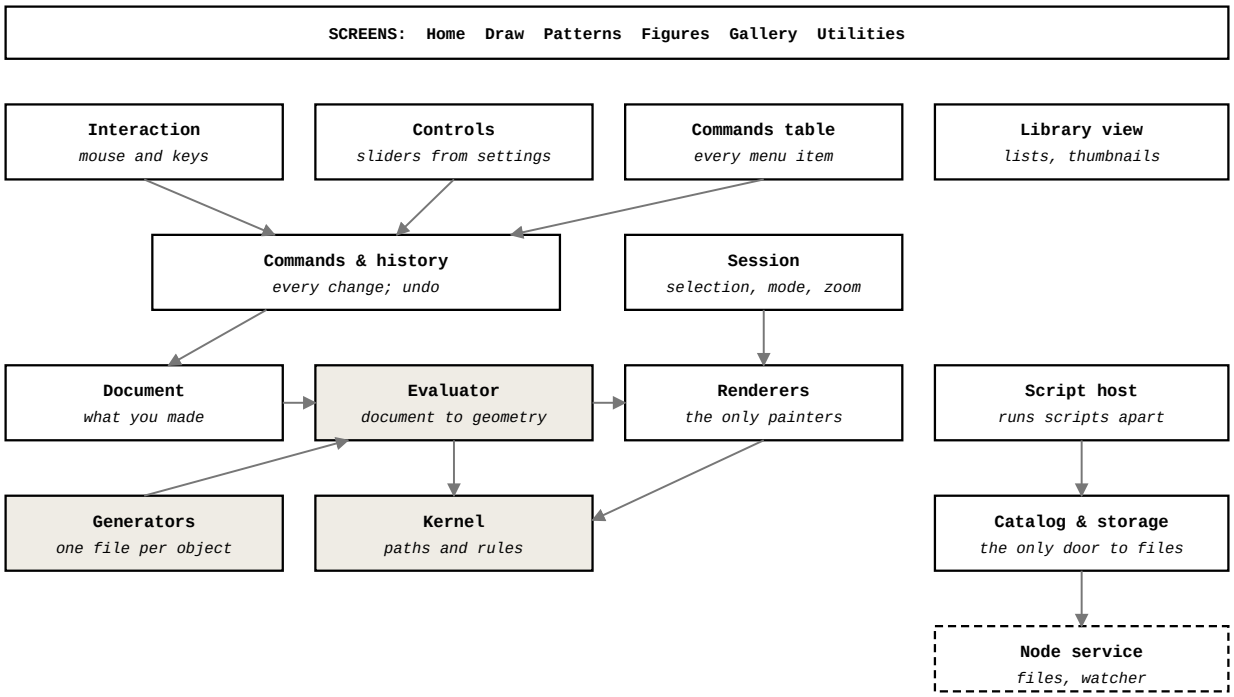


Figure 6. Cycle 2: the blocks

What you touch. The *screens* are the six tabs you know. *Interaction* is the one place mouse and keys are understood: it works out what is under the cursor and turns a gesture into a command. *Controls* builds sliders and boxes from an object's settings. The *commands table* holds every menu item and button in one list. The *library view* is today's category list builder, kept as it is.

What keeps track. *Commands and history* is the only way the document changes; it keeps a copy of the document before each change, which is what makes undo work. *Session* holds what you are looking at (selection, mode, zoom), which is never saved and never undone. The *document* is what you made.

The drawing model (shaded). The *evaluator* turns the document into geometry, layer by layer, using the *generators* and the *kernel*. The *renderers* are the only code that paints: screen, thumbnail, SVG, PDF.

Files and scripts. The *script host* runs each script in its own sealed room, so nothing carries over. *Catalog and storage* is the only door to files: the lists you see are worked out from the folders, so there is nothing to rebuild or keep in step. The *Node service* does the writing and watches the folders.

Cycle 3. Three actions, followed through the blocks

Dragging a handle. Figure 7 follows one drag on a Parabola layer whose corner another layer is attached to. Look at step 4: the drag changes one entry in the point table, and nothing else. Everything after that is recomputed from the document, which is why nothing can be left out of step.

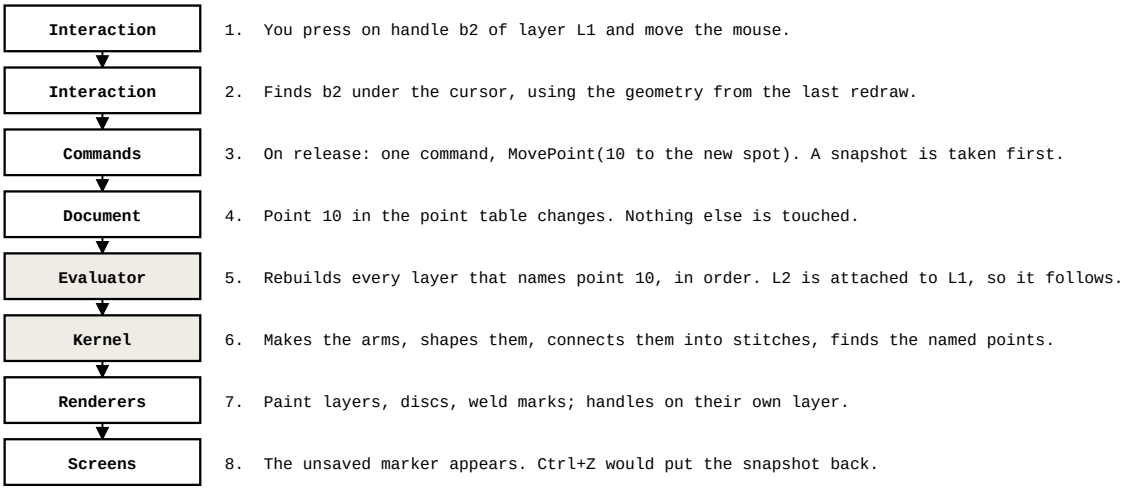


Figure 7. Dragging a handle, block by block

Adding a script. You save a file into a library folder. The Node service notices, reads the file's description without running it, and tells the open program, which adds the item to its list (Figure 8). Edit the file in VS Code and save again, and an open drawing reruns with your settings kept.

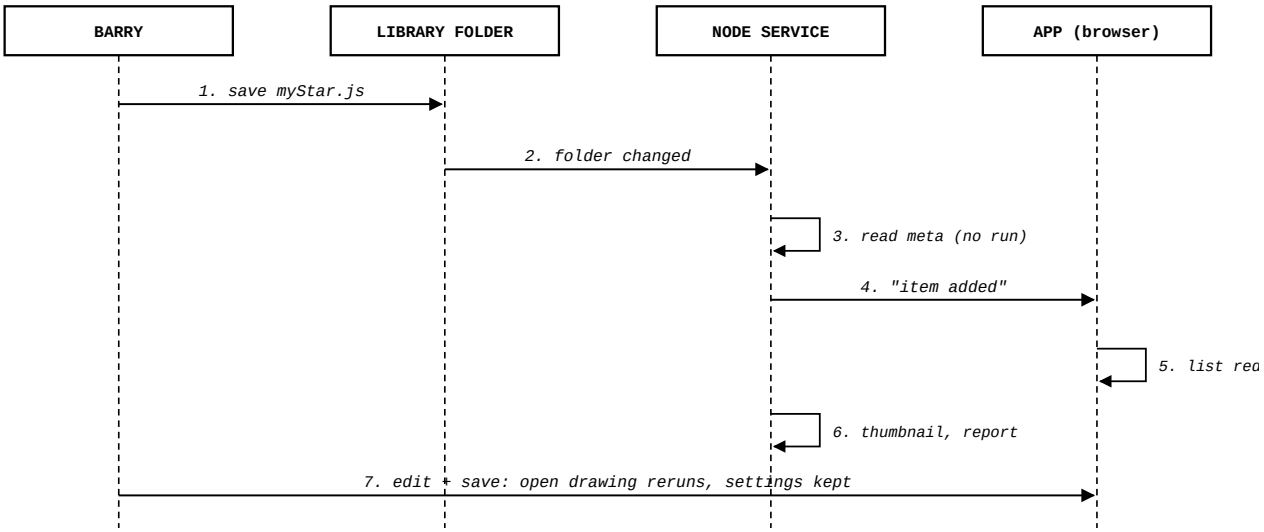


Figure 8. Adding a script by saving a file

Undo. Before each command, the whole document is copied onto the history (Figure 9). Undo puts the previous copy back; redo moves forward again. "Unsaved changes" just means you are not at the position where you last saved. Undo back to it, and the figure is clean again.

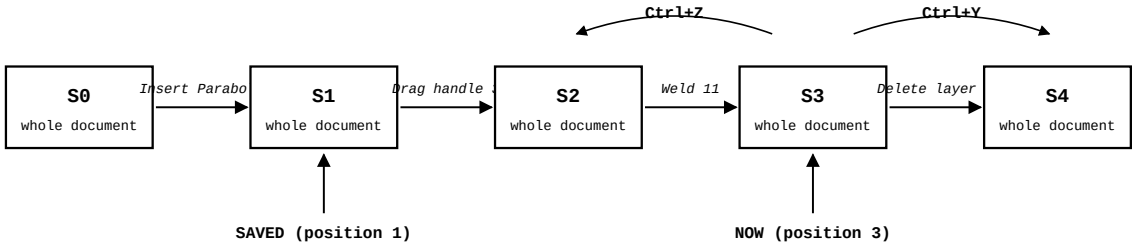


Figure 9. The history: one copy of the document per step

Part 4. The drawing model: a tutorial

4.0 The whole model on one page

Everything Sketchpad draws comes from four verbs:

- **Make** a path from a rule: a line, a circle, a rose.
- **Shape** it: space its points evenly, trim it, reverse it, turn it. Shaping can be repeated as often as you like.
- **Connect** one or more paths into stitch lines.
- **Find** the points worth naming: an end, a corner, the tip of a lobe.

Named points go into the document's point table. That is where nails come from, and where one layer's handles can be tied to another's.

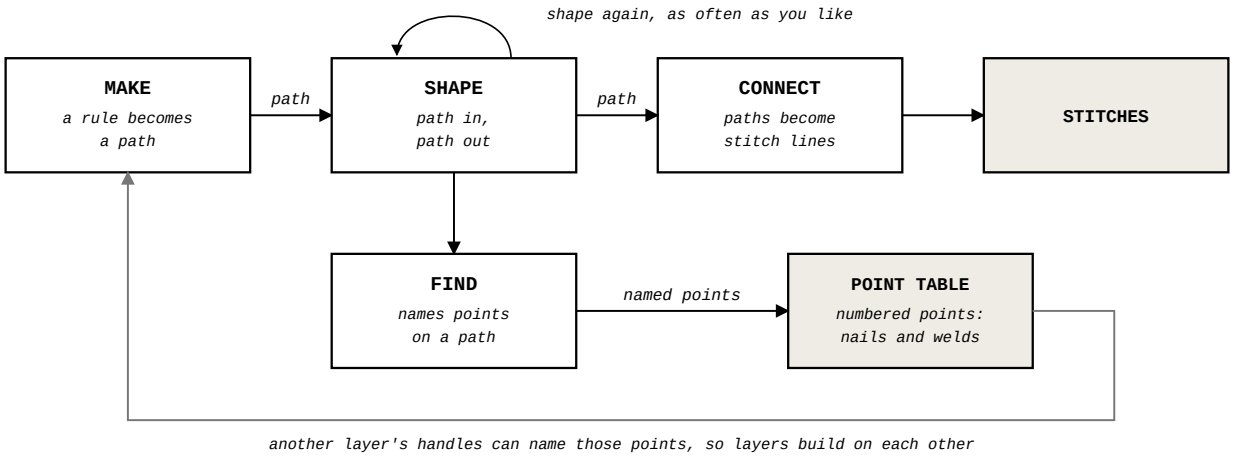


Figure 10. The drawing model: make, shape, connect, find

> **Vocabulary for this part.** A **path** is a numbered list of points. A **maker** builds one; a **shaper** turns one path into another; a **connector** turns paths into stitch lines; a **finder** names points on a path. The **point table** is the document's list of numbered points. A **generator** is one file that strings these together into an object such as the Parabola.

Lesson 1. A path is a list of points

Under every line Sketchpad draws there is an ordered list of points. A **path** is that list plus one fact: whether the last point joins back to the first. In Figure 11 the line is an open path of 9 points, numbered 0 to 8. The circle is a closed path of 12, numbered 0 to 11, and on a closed path counting past 11 comes round to 0 again. That wrap-around is what makes stars and roses possible (Lesson 5).

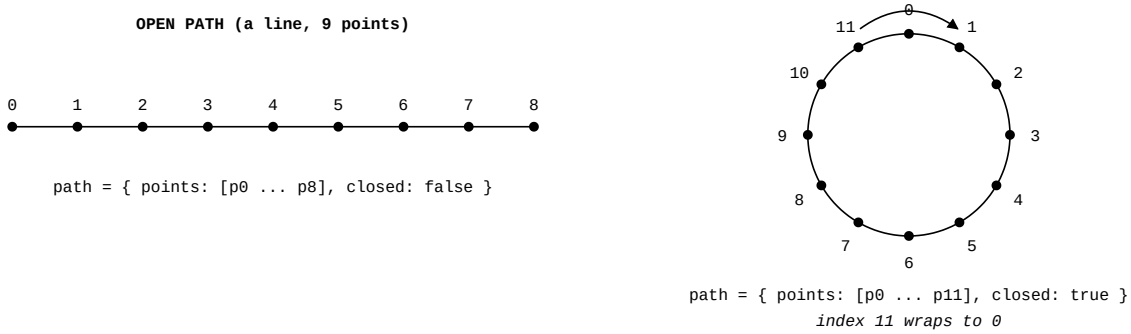


Figure 11. An open path and a closed path

What to notice. The numbers are positions in the list, not names. Point 3 is "the fourth one along", whatever its coordinates are. Every rule in this tutorial is a rule about those positions.

Lesson 2. Connecting two paths: the parabola

Take two lines that meet at a corner, each divided into 8 steps, so each is a path of 9 points. The connector `pairwise` joins point *i* of the first to point *i* of the second, for every *i* (Figure 12).

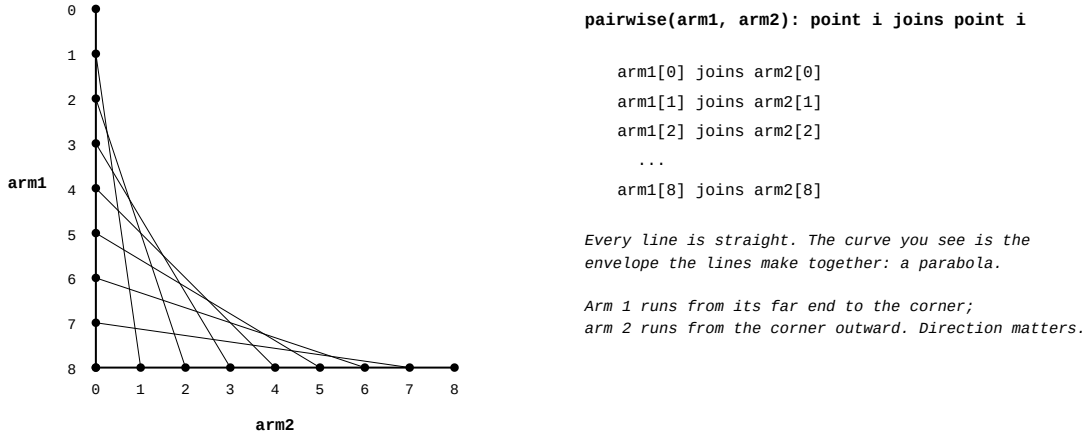


Figure 12. pairwise: point i joins point i

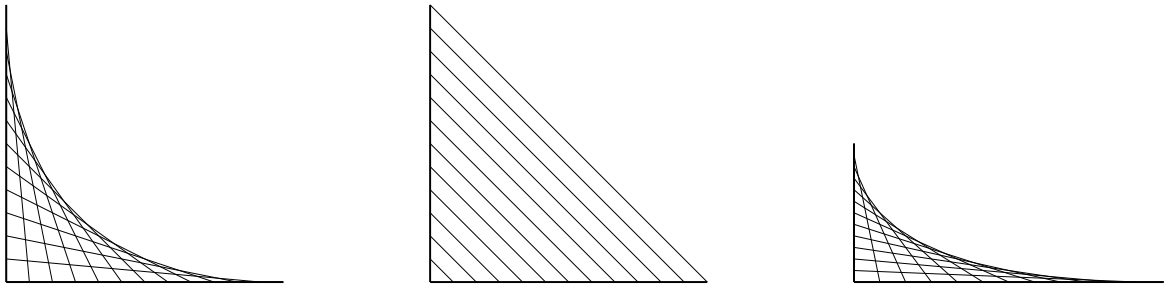
Every stitch is straight. The curve is what the eye sees where the stitches crowd together: the envelope of the lines. It is a true parabola. That's why string art can make curves from straight thread, and it is the whole of what the Parabola object does.

What to notice. The arms run in a particular direction: arm 1 from its far end towards the corner, arm 2 from the corner outward. Point 0 of arm 1 is the far end; point 0 of arm 2 is the corner. Lesson 3 shows why that matters.

Lesson 3. Direction and length matter

Leave the connector alone and change only the paths (Figure 13). Reverse arm 2, so it runs from its far end back to the corner, and every stitch becomes parallel to the others: no curve at all. Make one arm shorter than

the other and the curve leans. The connector never changed; only the lists it was given did.



`pairwise(arm1, arm2)`
a parabola

`pairwise(arm1, reverse(arm2))`
every line parallel: no curve at all

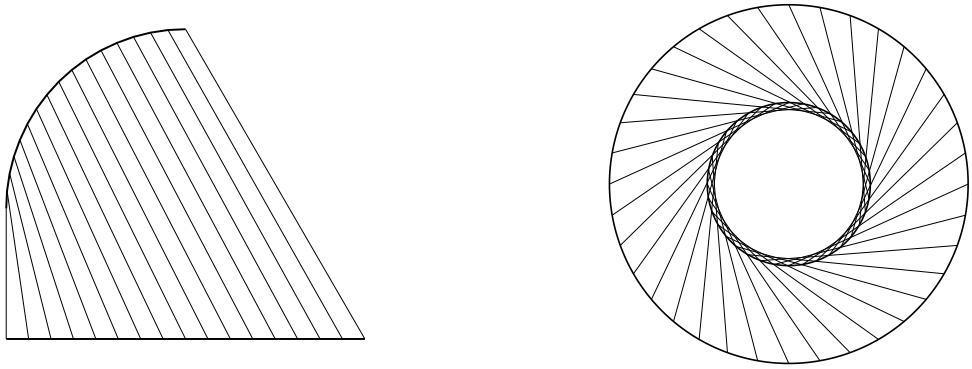
`pairwise(shortArm, longArm)`
unequal arms: a lopsided curve

Figure 13. Same connector, paths changed

Try it (on the kernel bench, checkpoint CP-1). Predict what `pairwise(reverse(arm1), arm2)` looks like before you draw it. It is the same as reversing arm 2, mirrored: parallel lines again.

Lesson 4. The same connector on other paths

Nothing in `pairwise` knows about lines. Give it an arc and a line and you get a curve swept between them. Give it two circles and you get a band. Start the small circle's numbering 9 places along (`startAt`, a shaper) and the band twists (Figure 14). This is the family Linked Circles belongs to: several circles, joined pairwise in a ring or all to all.



`pairwise(arc, line)`

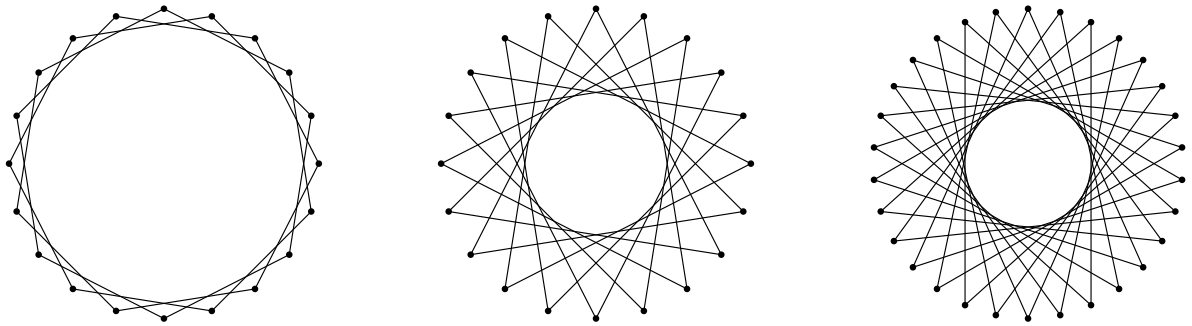
`pairwise(bigCircle, startAt(smallCircle, 9))`

Figure 14. pairwise between an arc and a line, and between two circles with a twist

Lesson 5. One closed path, joined to itself

A connector can join a path to itself. Three rules cover most of the classic string-art figures.

Skip. Join each point to the point *s* places further on (Figure 15). On a circle of 20 points with *s* = 7, point 0 joins 7, point 1 joins 8, and point 13 joins 20, which is past the end, so it wraps round to 0. The wrap-around is what closes the star. Chords in Ellipse is this rule on an ellipse.

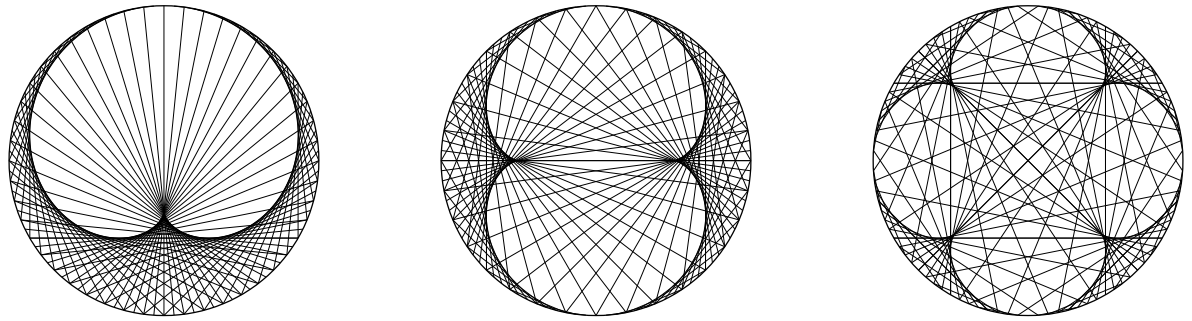


skip(circle of 20, 3) skip(circle of 20, 7) skip(circle of 30, 11)

point 0 joins 7, 1 joins 8, ... 13 joins 20, which is past the end, so it wraps to 0

Figure 15. skip: each point joins the point s places on

Multiply. Join point *i* to point *k*×*i* (Figure 16). With *k* = 2 on 96 points, point 1 joins 2, point 2 joins 4, and point 50 joins 100, which wraps round to 4. The result is a cardioid, which is what the Cycloid object draws. *k* = 3 gives a nephroid (two lobes), *k* = 5 four lobes. The number of lobes is always *k* − 1.



multiply(circle of 96, 2) multiply(circle of 96, 3) multiply(circle of 96, 5)

point i joins point k times i. With k = 2: 1 joins 2, 2 joins 4, 50 joins 100, which wraps to 4.

Figure 16. multiply: point i joins point k times i

All pairs. Join every point to every other point (Figure 17). That is the mystic rose. The number of lines grows quickly: *n* points give *n*(*n* − 1)/2 lines, so 14 points give 91 lines and 30 give 435. On a board that is 435 lengths of thread, which is worth knowing before choosing *n*.

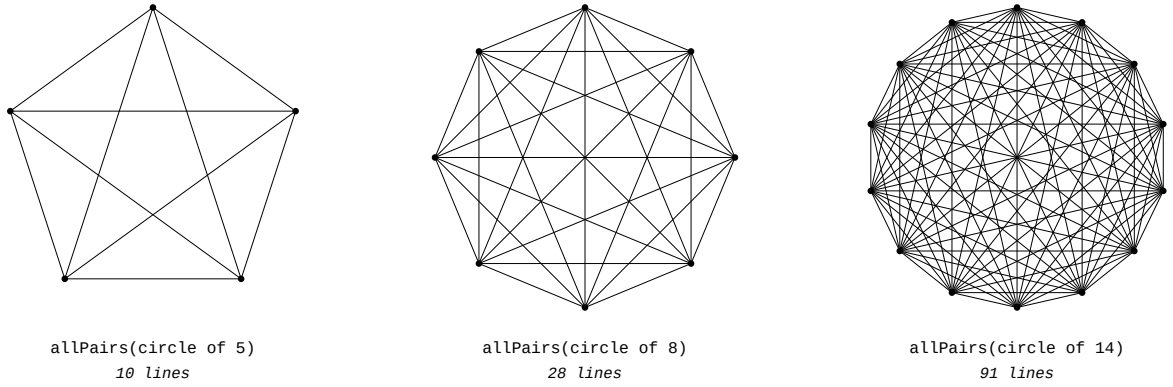


Figure 17. allPairs: every point to every other

Lesson 6. Shapers: changing a path before it is connected

A shaper takes a path and returns another. Your `createNodes` already did several of these, fused together and for ellipses only. Version 2 takes them apart so each works on any path, in any order (Figure 18).

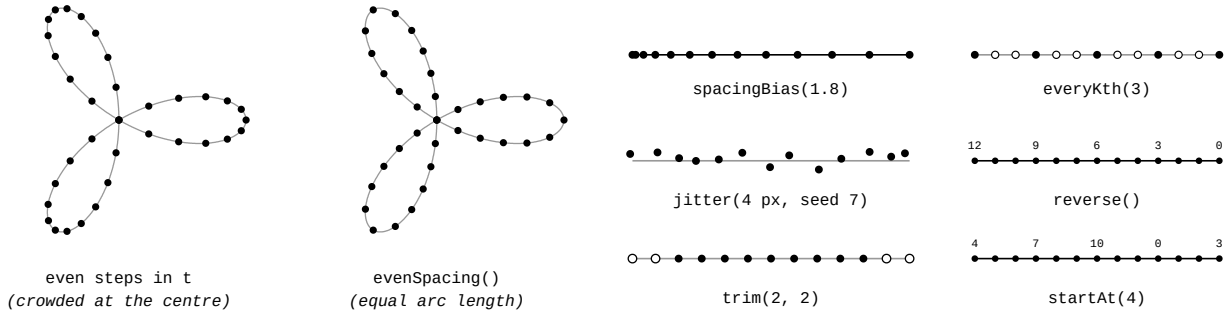


Figure 18. Shapers

Shaper	What it does	Typical use
`evenSpacing(path, n)`	Re-places n points at equal distances along the path	Nails at even spacing on any curve
`bias(path, b)`	Crowds points towards one end	Denser stitching where the eye looks
`jitter(path, size, seed)`	Moves each point a little, at random but repeatably	A hand-made look; the same seed gives the same picture
`trim(path, a, b)`	Drops a points from the start and b from the end	Today's Trim and Shorten sliders
`reverse(path)`	Runs the list backwards	Lesson 3
`startAt(path, i)`	Renumbers a closed path to start at point i	Twists (Lesson 4); turns a star without moving it
`everyKth(path, k)`	Keeps every k-th point	Fewer nails on the same curve

`move, rotate, scale`	Moves the points	Placing a path; a layer's own move and turn

What to notice. Jitter takes a seed. With the same settings and the same seed, the same picture comes back every time, which matters for a design you mean to print.

Lesson 7. Where paths come from: makers, and why parametric is not special

A maker is a rule that says where the point is at each moment *t*, as *t* runs from start to finish. Sampled at *n* moments, it gives a path. A line is the simplest rule: start at *a* and move steadily towards *b*. A circle goes round its centre. A Lissajous figure or a rose is just a longer formula (Figure 19). Seen this way, a "parametric drawing" is an ordinary path from an unusual maker, and everything in Lessons 2 to 6 applies to it.

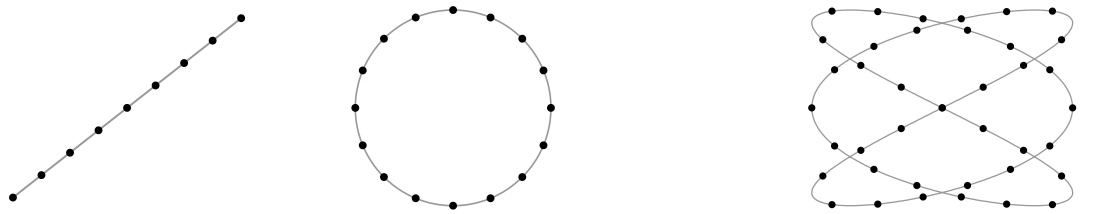


Figure 19. Three makers, one kind of result

One difference is real. Taking equal steps in *t* does not give equal spacing along a curve. On a rose, equal steps crowd the points near the centre (left of Figure 18). Nails want equal spacing, so a curve is usually followed by `evenSpacing`. Version 2 also keeps two samplings of a curve (Figure 20): a fine one, with many points, for drawing it smoothly on screen, and a coarse one, with one point per nail, for stitching and printing.

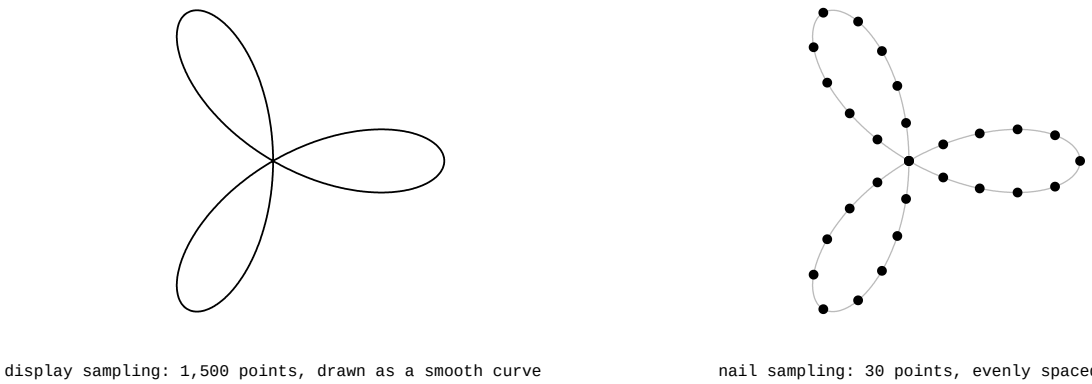


Figure 20. The same curve, sampled for display and for nails

Lesson 8. Finding points worth naming

A finder looks at a path and names points on it (Figure 21): its ends and middle, its extremes, the tips of a rose's lobes, its cusps, the places where two paths cross. These are Phase 8b's "magic points", and they are what other layers attach to.

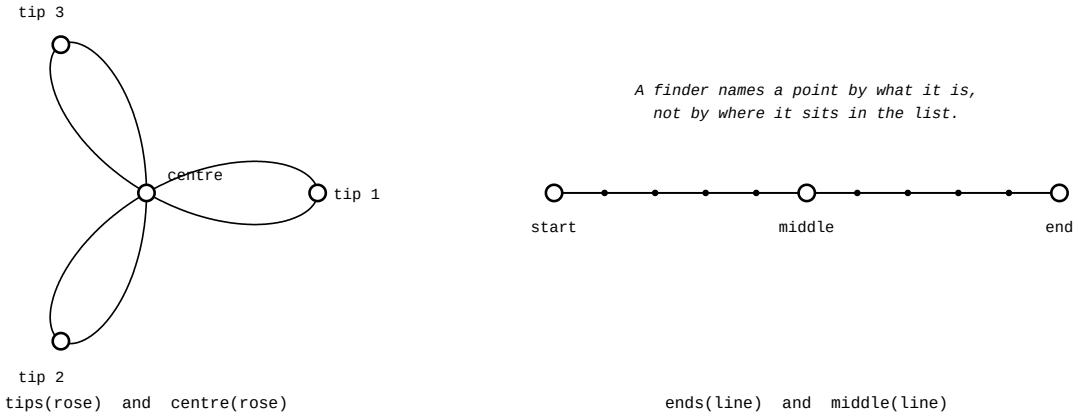


Figure 21. Finders name points by what they are

What to notice. A finder names a point by what it is ("tip of lobe 2"), not by its position in the list ("point 17"). Change the rose from 30 points to 40 and point 17 is somewhere else, but tip 2 is still tip 2. That is what keeps attachments stable when you change a step count.

Lesson 9. Layers that use other layers

So far each drawing has made its own paths. In a figure, a layer can use paths that belong to other layers. In Figure 22, layer L3 owns no shape: it connects layer L1's circle to layer L2's line with `pairwise`. Move L1 or L2 and L3 follows, because it is recomputed from them.

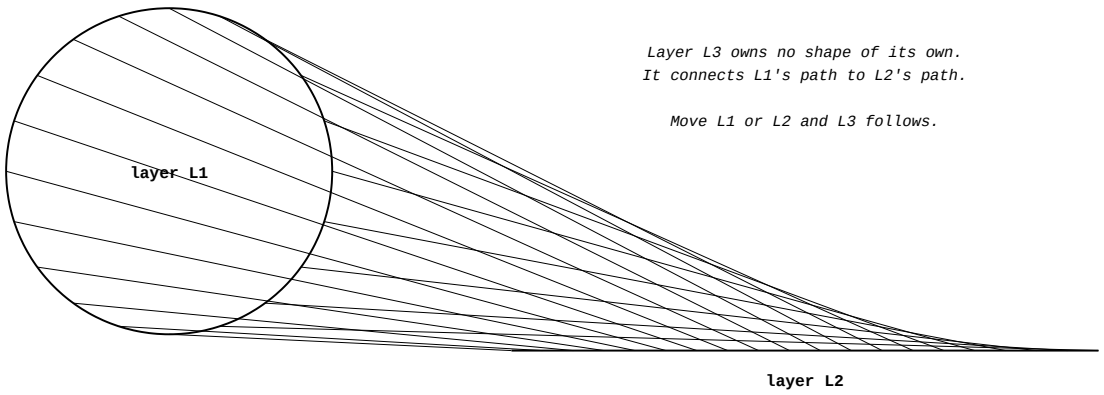
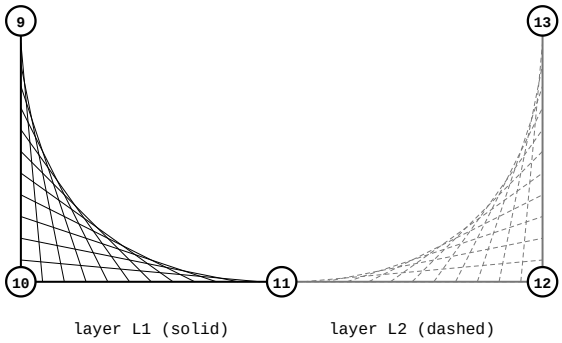


Figure 22. A layer that connects two other layers' paths

The same idea joins handles. Every handle names a numbered point in the document's **point table**, and a table entry is one of two kinds.

A free point holds its own position. Two handles that name the same free point are welded: there is one point and two names for it (Figure 23). Drag it and both layers redraw. There is nothing to copy and nothing to keep in step. This is your rule from October: a weld is a shared point number.



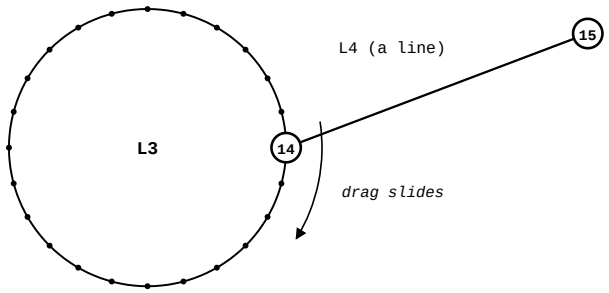

```
points
 9: (60, 30)
10: (60, 190)
11: (220, 190)
12: (380, 190)
13: (380, 30)
```

```
layers
L1: parabola
  line1_pt1  -> 9
  line1_pt2  -> 10
  line2_pt1  -> 10
  line2_pt2  -> 11
L2: parabola
  line1_pt1  -> 11
  line1_pt2  -> 12
  line2_pt1  -> 12
  line2_pt2  -> 13
```

Point 11 is stored once. Both layers name it.
Drag it and both redraw. Nothing is copied.

Figure 23. Two layers name point 11

An attached point doesn't hold a position. It says "a quarter of the way along layer L3's path", or "on the tip of lobe 2 of layer L5". The other layer decides where it is, and the handle follows (Figure 24). It works one way only: dragging the attached handle slides it along the path; it never pulls the other layer out of shape. Anchored discs, a handle dropped mid-arm, and anything placed on scaffold all work like this.



L4 follows L3. L3 never follows L4.

```
points
14: { on: "L3", t: 0.25 }    attached
15: (420, 50)                free

L4: line
  pt1 -> 14
  pt2 -> 15
```

Figure 24. Point 14 lies on layer L3's path

Because attachments run one way, the layers can always be worked out in order: each one after the layers it is attached to (Figure 25). If you try to attach L6 to L5 when L5 is already attached to L6, there is no order that works, so Sketchpad refuses at that moment, with a message.

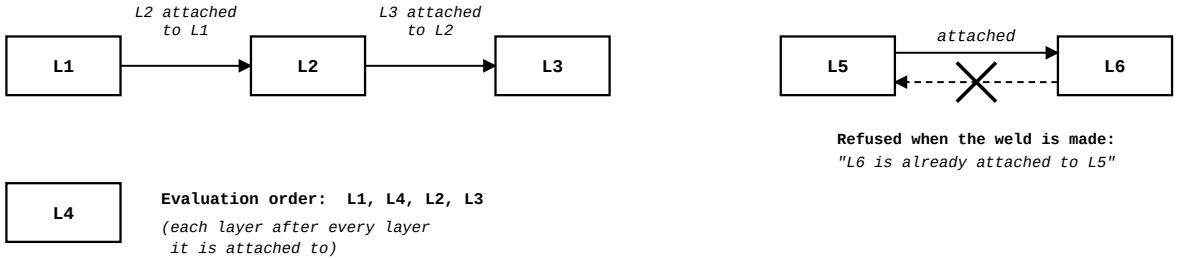
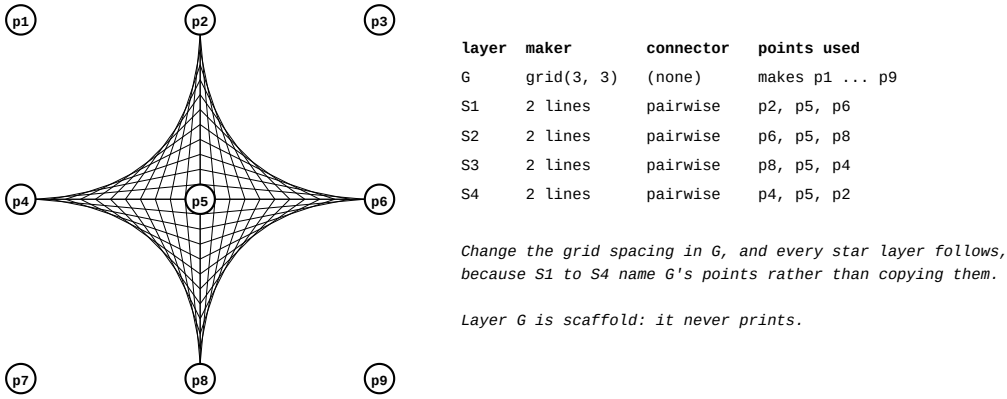


Figure 25. Layers are worked out in order; a loop is refused

Lesson 10. Putting it together: a figure with no code

Many of the pattern scripts are a figure written out as code. `OverlappingStars.js` works out twenty named points on a grid, then stitches parabolas between groups of them. In Version 2 that kind of drawing can be put together from layers instead (Figure 26). Layer G makes a 3-by-3 grid of named points and is marked as scaffold, so it never prints. Four Parabola layers each name three of G's points: two arm ends and the shared centre p5. The result is a four-pointed star.



layer	maker	connector	points used
G	grid(3, 3)	(none)	makes p1 ... p9
S1	2 lines	pairwise	p2, p5, p6
S2	2 lines	pairwise	p6, p5, p8
S3	2 lines	pairwise	p8, p5, p4
S4	2 lines	pairwise	p4, p5, p2

Change the grid spacing in G, and every star layer follows, because S1 to S4 name G's points rather than copying them.

Layer G is scaffold: it never prints.

Figure 26. A star composed from a grid, with no code

What to notice. Change the spacing of grid G and the star redraws at the new size, because its layers name G's points instead of copying them. Add a second grid offset by half a cell, with four more layers, and you have overlapping stars: the same idea as the script, but you can now drag it, weld it and print it.

Lesson 11. Writing your own generator

When composition isn't enough, you write a generator: one file, saved into `generators/`. It appears in the Insert list straight away. Listing 1 is a complete one that doesn't exist today: a spiral with each point joined to the point a fixed number of steps further along.

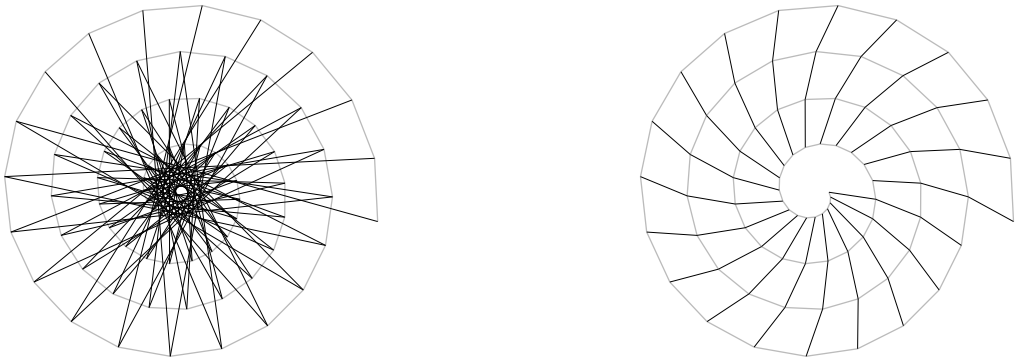
```
 1 /* generators/spiralWeb.js -- a spiral, each point joined to the point "gap" steps on. */
 2 export const meta = { id: "spiralWeb", version: 1, title: "Spiral Web",
 3                       tags: ["stitch", "spiral"], notes: "Skip-connected spiral." };
 4
 5 export const params = {
 6   centre: { type: "point", handle: true, default: [400, 400] },
 7   turns:  { type: "number", default: 4,  min: 1, max: 10, step: 0.5, label: "Turns" },
 8   steps:  { type: "int",    default: 80, min: 20, max: 300, label: "Points" },
 9   gap:    { type: "int",    default: 9,  min: 1, max: 60,  label: "Gap" },
10   size:   { type: "number", default: 120, min: 20, max: 380, label: "Size" },
11   color:  { type: "color",  default: "#0044cc" }
12 };
13
14 export function build(p, k) {
15   const spiral = k.spiral(p.centre, { turns: p.turns, outer: p.size }, p.steps);
16   return { paths: { spiral },
17            stitches: k.skip(spiral, p.gap, { wrap: false }),
18            points: { centre: p.centre, outerEnd: k.ends(spiral).end },
19            style: { color: p.color } };
20 }
```

Listing 1. A complete generator

Line by line:

- **Lines 2 to 3** name it. `id` is permanent, because saved figures refer to it.
- **Lines 5 to 12** declare the settings. Each says what kind of value it is, and the sliders are built from that. `centre` is a draggable point because it says `handle: true`.
- **Line 15** makes the path.
- **Line 17** connects it. The spiral is open, so `wrap: false` stops the skip from running past the end.
- **Line 18** names two points that other layers may attach to.

There is no `init`, `update` or `draw`, and nothing is stored between calls. Figure 27 shows the output with two values of Gap.



spiralWeb: steps 80, gap 9 the same, gap 20

Figure 27. Spiral Web, as Listing 1 draws it

Lesson 12. How today's objects come out

Each of the 14 Draw objects becomes a short recipe. The ones marked *draft* are first guesses, to be settled when each object's one-page feature list is written with you (specification 10.2).

Object	Recipe
Line	`line(a, b)`; the middle handle moves both ends
Parabola	two `line` arms, `pairwise`; the corner is two handles until you weld them
Regular Polygon	`polygon`; `pairwise` at each corner between neighbouring sides
Mystic Rose, Mystic Rose (Ellipse)	`circle` or `ellipse`, `allPairs`
Cycloid	`circle` (or ellipse), `multiply(k)`
Chords in Ellipse	`ellipse`, `skip(s)`; *draft*: how its five modes map, from its feature list
Circular Parabola	*draft*: the Parabola recipe, repeated by rotation around a centre
Circular Parabola (Multi)	as above; its second handle separates and joins the copies through the generator's own `drag` rule
Linked Circles	several `circle` paths, joined `pairwise` as pairs, a ring, or all to all
Parametrics	`parametric`, `evenSpacing`, then any connector; what can be dragged is still to decide
Inverse Star, Radiate, Regular Pursuit	*draft*: from their feature lists

Lesson 13. What the model does not cover

The model is about points and lines, because nails and thread are. Fills, gradients, transparency and pixel effects don't fit it, and don't need to. Playpen scripts that work that way keep their own canvas inside the script host and paint directly, as they do today. They can still be shown, given thumbnails, and placed in a figure as a picture. They just can't give nails.

Quick reference: the kernel's vocabulary

Family	Name	Meaning
Makers	`line(a, b, n)`	n steps from a to b
	`circle(c, r, n)`, `ellipse(c, w, h, n)`	n points round, closed
	`polygon(c, r, sides, n)`	sides with n steps each, closed
	`arc(c, r, from, to, n)`	part of a circle, open
	`spiral(c, options, n)`	a spiral, open
	`parametric(f, t0, t1, n)`	any rule p(t), sampled
Shapers	`evenSpacing(path, n)`	equal distances along the path
	`bias(path, b)`	crowd towards one end
	`jitter(path, size, seed)`	repeatable random nudges
	`trim(path, a, b)`	drop points from the ends
	`reverse(path)`	run backwards
	`startAt(path, i)`	renumber a closed path
	`everyKth(path, k)`	keep every k-th point
	`move`, `rotate`, `scale`, `mirror`, `split`, `join`	placement and assembly
Connectors	`pairwise(p, q)`	point i of p to point i of q
	`skip(path, s)`	point i to point i + s
	`multiply(path, k)`	point i to point k × i
	`allPairs(path)`	every point to every other
	`ring(paths, rule)`	each path to the next, round
	`between(paths, rule)`	several paths, pairs or all to all
Finders	`ends`, `middle`	start, end, halfway
	`extremes`	top, bottom, left, right
	`tips`, `cusps`, `centre`	features of curves
	`crossings(p, q)`	where two paths cross

Closed paths wrap round: counting past the last point starts again at 0. When two paths of different lengths are connected, the connector either respaces one to match, or stops with a message that names both lengths and the missing point.